Testing de la API de Autenticación con Postman

Análisis y Desarrollo de Software

Servicio Nacional de Aprendizaje (SENA)

Eliana Chacon

1 de noviembre de 2026

1. Introducción

Esta evidencia prueba con Postman la API construida en AA5-EV01: el servicio de registro e inicio de sesión `autenticacion-pergamino`.

La colección tiene **27 peticiones con 44 comprobaciones**, y lo que vigila no es principalmente que el servicio responda. Es que **no responda de más**: que un correo inexistente y una contraseña equivocada den exactamente la misma respuesta, que el resumen de la contraseña no aparezca nunca en el cuerpo, y que el freno por intentos fallidos no se le anuncie a quien no sabe la contraseña. Esas son las propiedades que un cambio bienintencionado rompe sin darse cuenta.

2. Objetivo

Probar la API de AA5-EV01 con Postman: instalar la herramienta, construir la colección de pruebas, documentar el resultado con pantallazos, grabar el video del testing y entregar el archivo de endpoints.

3. Instalación de Postman

Postman se instala desde postman.com/downloads. En Linux, la vía más corta es el paquete snap:

```
$ sudo snap install postman
postman 11.x from Postman, Inc. installed

$ postman
```

Captura 1 — Postman instalado

La ventana de Postman recién abierta, con el área de trabajo vacía

4. La colección

El archivo está en el repositorio y se importa con **File → Import**:

```
entregas/GA7/AA5_EV02_Postman_API/postman/
    MONK_GRAIN_Autenticacion.postman_collection.json
```

Captura 2 — Colección importada

El panel izquierdo mostrando «MONK GRAIN — Autenticación (AA5-EV01)» con sus cuatro carpetas desplegadas

4.1 Las variables, y por qué no hay nada escrito a mano

Variable	Valor	Para qué
`baseUrl`	`http://localhost:8082/api/auth`	El puerto 8080 es del catálogo (AA2-EV02) y el 8081 de la negociación (AA3-EV01)
`email`	(se genera)	Un correo al azar en cada ejecución
`password`	`pergamino2026`	13 caracteres, con letras y números: cumple la política
`token`	(se guarda)	Lo escribe la prueba de ingreso, no la persona

El correo se genera al azar en el guion previo de la primera petición. Sin eso, la segunda ejecución de la colección chocaría con la cuenta de la primera y, lo que es peor, con su freno de quince minutos:

```
// Un correo distinto en cada ejecución: así la colección se puede correr
// muchas veces sin chocar con la cuenta de la vez anterior, ni con su
// freno de quince minutos.
const sufijo = Math.floor(Math.random() * 1000000);
pm.collectionVariables.set('email', `ana.restrepo.${sufijo}@finca.co`);
```

El token tampoco se copia y pega. La petición 2.1 lo guarda y las ocho peticiones de la carpeta 3 lo usan desde la variable. Copiarlo a mano haría la colección imposible de ejecutar de corrido.

4.2 Las cuatro carpetas

Carpeta	Peticiones	Qué verifica
1. Registro	6	Que el acuse sea el mismo exista o no el correo
2. Ingreso	5	Que los dos modos de fallar respondan idéntico
3. Sesión	9	Que el token sirva, y que se pueda revocar
4. Freno	7	A quién se le explica el freno y a quién no

Se ejecutan **en orden** y cada carpeta depende de la anterior: la 3 usa el token que guardó la 2, y la 4 gasta los cinco intentos sobre la cuenta que creó la 1. No es una colección de peticiones sueltas, es un recorrido.

5. Las pruebas, una por una

5.1 Registro

#	Petición	Esperado	Qué protege
1.1	Cuenta nueva	`202`	Que no sea `201`: anunciaría que se creó algo
1.2	El **mismo** correo	`202` idéntico	Compara los cuerpos byte por byte
1.3	Contraseña «corta»	`422`, 2 motivos	Que diga de una vez todo lo que falta
1.4	Correo mal formado	`400`	Que diga qué campo está mal
1.5	JSON con una coma de más	`400`	Que no salga como `500`
1.6	Pidiendo `"role":"ADMIN"`	`202`, ignorado	Asignación masiva

La comprobación que da sentido a toda la carpeta es la de 1.2:

```
pm.test('El cuerpo es idéntico al de la cuenta nueva', function () {
    // Byte por byte. Si se diferenciaran en algo —el mensaje, un campo
    // extra, el código de estado— el registro serviría para averiguar qué
    // correos están registrados en MONK GRAIN.
    pm.expect(pm.response.text()).to.eql(pm.collectionVariables.get('acuseRegistro'));
});
```

Captura 3 — Registro con correo nuevo (1.1)

La respuesta `202 Accepted` y la pestaña **Test Results** con las dos comprobaciones en verde

Captura 4 — El mismo correo repetido (1.2)

La respuesta, idéntica a la de la captura anterior, y la comprobación «El cuerpo es idéntico al de la cuenta nueva» en verde

Captura 5 — Contraseña débil (1.3)

La respuesta `422` con los dos motivos en `details`

5.2 Ingreso

#	Petición	Esperado	Qué protege
2.1	Credenciales correctas	`200` + token	Que no se filtre `$2a$` en el cuerpo
2.2	Segundo ingreso	token distinto	Que cada dispositivo tenga su sesión
2.3	Contraseña equivocada	`401` neutral	—
2.4	Correo inexistente	**el mismo** `401`	Enumeración de cuentas
2.5	Sin contraseña	`400`	Error de forma, no de autenticación

La comprobación de 2.4 es la que da sentido a todo el servicio, y está escrita para ponerse roja si alguien «mejora la experiencia»:

```
pm.test('La respuesta es idéntica a la de la contraseña equivocada', function () {
    // ESTA es la prueba que da sentido a todo el bloque. Si alguien
    // «mejora la experiencia» separando los dos mensajes, se pone roja.
    const sinFecha = pm.response.json();
    delete sinFecha.timestamp;
    pm.ex-
pect(JSON.stringify(sinFecha)).to.eql(pm.collectionVariables.get('fallo_contrasena'));
});
```

La marca de tiempo se quita porque es lo único que cambia legítimamente entre dos respuestas. Todo lo demás debe coincidir.

Y en 2.1, dos comprobaciones sobre lo que no debe salir:

```
pm.test('La respuesta NO lleva el resumen de la contraseña', function () {
    // Se busca en el cuerpo completo y no solo en el campo esperado: así
    // también se detecta que aparezca donde nadie lo buscaba.
    pm.expect(pm.response.text()).to.not.include('$2a$');
    pm.expect(pm.response.json().account.passwordHash).to.be.undefined;
});
```

Captura 6 — Ingreso correcto (2.1)

La respuesta `200` con `"message": "Autenticación satisfactoria."` y el token, con las cinco comprobaciones en verde

Captura 7 — Los dos fallos, lado a lado (2.3 y 2.4)

Las respuestas de la contraseña equivocada y del correo inexistente, para que se vea que son el mismo `401` con el mismo cuerpo

5.3 Sesión

#	Petición	Esperado
3.1	`GET /me` con el token	`200`, y `role: BUYER`
3.2	Sin cabecera	`401`
3.3	Token inventado	`401`
3.4	Esquema `Basic`	`401`
3.5	`bearer` en minúsculas	`200`
3.6	Cerrar sesión	`200`
3.7	`GET /me` con el token cerrado	`401`
3.8	`GET /me` con el token del otro dispositivo	`200`
3.9	Cerrar sesión otra vez	`200`

La pareja 3.7 / 3.8 es la que justifica haber elegido sesiones revocables en lugar de JWT: el token cerrado deja de valer al instante, y el del otro dispositivo sigue abierto. Con un JWT, la firma del primero seguiría siendo válida hasta la caducidad.

Y 3.1 merece una nota aparte. Es la petición que encontró un error `500` real en el servicio, con las cuarenta pruebas de JUnit en verde. El detalle está en el informe de AA5-EV01, §6; en resumen, las pruebas corrían dentro de una transacción que mantenía abierta la sesión de Hibernate, y en producción no hay tal transacción.

Captura 8 — El perfil con el token (3.1)

La respuesta `200` con los datos de la cuenta, y la cabecera `Authorization` visible en la pestaña **Headers**

Captura 9 — El token cerrado ya no abre nada (3.7)

La respuesta `401` usando el mismo token que acababa de funcionar

5.4 Freno por intentos fallidos

#	Petición	Esperado
4.1 – 4.5	Cinco contraseñas equivocadas	`401` neutral las cinco
4.6	La contraseña **correcta**	`429` + `Retry-After`
4.7	La contraseña **equivocada**, ya frenada	`401` neutral

4.6 y 4.7 son la misma cuenta en el mismo estado, y responden distinto según quién pregunta. Es el punto más fino del diseño:

```
cuenta frenada + contraseña CORRECTA  → 429  «vuelve en 15 minutos»
cuenta frenada + contraseña EQUIVOCADA → 401  «correo o contraseña incorrectos»
```

Si el freno se anunciara a cualquiera, provocarlo bastaría para confirmar que el correo existe, y toda la neutralidad de la carpeta 2 se caería por aquí. Quien acierta la contraseña, en cambio, ya demostró ser el dueño: a él sí se le dice la verdad completa.

Captura 10 — La cuenta frenada, con la contraseña correcta (4.6)

La respuesta `429`, el mensaje con los minutos que faltan, y la cabecera `Retry-After` en la pestaña **Headers**

Captura 11 — La misma cuenta frenada, con la contraseña mala (4.7)

La respuesta `401` neutral, que no delata el freno

6. Ejecución completa

La colección se ejecuta de corrido con el **Collection Runner** (botón Run sobre la colección), que corre las 27 peticiones en orden y muestra el resultado de las 44 comprobaciones.

El mismo recorrido desde la consola, con `newman`, que es Postman sin ventana:

```
$ newman run MONK_GRAIN_Autenticacion.postman_collection.json

┌─────────────────────────┬────────────────────┬────────────────────┐
│                         │           executed │             failed │
├─────────────────────────┼────────────────────┼────────────────────┤
│              iterations │                  1 │                  0 │
│                requests │                 27 │                  0 │
│            test-scripts │                 27 │                  0 │
│      prerequest-scripts │                  1 │                  0 │
│              assertions │                 44 │                  0 │
├─────────────────────────┴────────────────────┴────────────────────┤
│ total run duration: 2.3s                                          │
│ total data received: 3.98kB (approx)                              │
│ average response time: 64ms [min: 6ms, max: 161ms, s.d.: 59ms]    │
└───────────────────────────────────────────────────────────────────┘
```

Captura 12 — El Collection Runner terminado

El resumen de la ejecución con las 44 comprobaciones en verde y ningún fallo

6.1 Un dato escondido en el resumen

`min: 6ms, max: 161ms`. Esa diferencia no es ruido: las respuestas de 6 ms son las que fallan la validación de forma y nunca llegan a comparar contraseñas; las de 120 ms o más son las que sí pasan por BCrypt.

Y ahí está la razón de una línea del servicio que de otro modo parecería inútil. Cuando el correo no existe, el servicio compara igualmente contra un resumen inventado:

```
correo REGISTRADO, contraseña mala:   124,5 ms
correo INEXISTENTE:                   119,6 ms   ← 4 % de diferencia

campo faltante (no llega a BCrypt):     4,9 ms   ← esto es lo que se vería
                                                   sin el resumen inventado
```

Sin esa comparación de relleno, un correo desconocido respondería en 5 ms y uno registrado en 120, y bastaría **cronometrar** para separar los correos que existen de los que no, aunque el mensaje fuera idéntico. Las medidas son de doce peticiones de cada clase sobre el servicio en marcha.

7. Resultado

Carpeta	Peticiones	Comprobaciones	Fallos
1. Registro	6	11	0
2. Ingreso	5	12	0
3. Sesión	9	11	0
4. Freno	7	10	0
Total	**27**	**44**	**0**

8. Guion del video

Entre ocho y diez minutos. El servicio debe estar corriendo antes de empezar a grabar: `java -jar target/autenticacion-pergamino.jar`.

Bloque	Minutos	Qué mostrar y qué decir
1. Presentación	0:00 – 0:45	Nombre, programa, y qué se va a probar: la API de registro e inicio de sesión de MONK GRAIN, cuatro endpoints en el puerto 8082.
2. Postman y la colección	0:45 – 1:45	Postman abierto. Mostrar la colección importada, las cuatro carpetas, y las variables. Decir por qué el correo se genera al azar: para poder correrla muchas veces sin chocar con el freno de la vez anterior.
3. Registro	1:45 – 3:15	Ejecutar 1.1 y señalar el `202`. Ejecutar 1.2 con el mismo correo y **detenerse ahí**: la respuesta es idéntica. Explicar por qué —un «ese correo ya existe» convertiría el registro en un buscador de cuentas—. Luego 1.3, la contraseña débil, y señalar que aquí sí se explica el motivo porque la política es pública.
4. Ingreso	3:15 – 4:45	Ejecutar 2.1 y mostrar el token y el mensaje «Autenticación satisfactoria.». Señalar que la respuesta no trae el resumen de la contraseña. Luego 2.3 y 2.4 seguidas, comparando las dos respuestas en pantalla: mismo código, mismo cuerpo.
5. Sesión	4:45 – 6:15	Ejecutar 3.1 y mostrar que el token abre el perfil. Luego 3.6, cerrar sesión, y 3.7 con el mismo token: ya no vale. Y 3.8, que el otro dispositivo sigue dentro. Decir aquí por qué no se usó JWT: su firma seguiría valiendo después de cerrar sesión.
6. El freno	6:15 – 8:00	Ejecutar las cinco peticiones fallidas seguidas. Luego 4.6, con la contraseña buena: `429` y `Retry-After`. Y por último 4.7, con la mala: `401` neutral. Explicar la diferencia: a quien no sabe la contraseña nunca se le dice que la cuenta está frenada, porque eso confirmaría que el correo existe.
7. La colección completa	8:00 – 9:00	Ejecutar el Collection Runner de principio a fin y mostrar las 44 comprobaciones en verde.
8. Cierre	9:00 – 9:30	Decir dónde está todo en el repositorio: la colección en `entregas/GA7/AA5_EV02_Postman_API/postman/` y el catálogo de endpoints en `ENDPOINTS.md`.

> **Lo que te falta hacer a ti**
>
> **Las doce capturas** señaladas a lo largo del documento, cada una con lo que debe verse descrito al lado.
>
> **El video de 8 a 10 minutos**, con el guion de la sección 8.
>
> Para las dos cosas, el servicio se levanta con `java -jar target/autenticacion-pergamino.jar` desde `entregas/GA7/AA5_EV01_API_Registro_Login/autenticacion-pergamino/`, y responde en **http://localhost:8082/api/auth**. La colección ya está construida y verificada: solo hay que importarla y ejecutarla.

9. Conclusión

Las 27 peticiones pasan sus 44 comprobaciones. Lo que importa de esta colección no es el número, sino qué se eligió comprobar: **22 de las 44** —la mitad justa— verifican que el servicio no haga algo: que no distinga los dos modos de fallar, que no publique el resumen de la contraseña, que no conceda el rol que la petición pidió, que no anuncie el freno a quien no sabe la contraseña.

Son propiedades que se rompen con un cambio razonable y bienintencionado: separar dos mensajes de error para «ayudar al usuario» es exactamente lo que convierte un inicio de sesión en una herramienta de reconocimiento. Por eso están escritas como pruebas, y no como una nota en un documento.